

Round Robin (RR)

Round Robin gives ready processes turns on the CPU. An unfinished process returns to the queue tail when its turn ends.

1. Service time and time slice

The visualizer uses one CPU, whole ticks, and CPU-only processes. Switching processes takes no time. **Service time** is the total CPU work needed to finish. The **quantum**, or **time slice**, limits CPU ticks per turn. The default quantum is 2; any positive whole number is allowed. RR has no separate allotment or priority levels.

A ready process has arrived and is waiting for the CPU. A future process has not arrived yet. Waiting does not consume service or quantum.

Queue entries read from left to right; [] means empty. Each timeline cell is one tick, and a dash (-) means the CPU is idle. Examples under different rules are separate scenarios.

2. Scheduling rules

Rule 1. Add arrivals to the queue tail

Existing waiters stay ahead of new arrivals. Simultaneous arrivals enter in input order. If arrival and quantum expiry coincide, admit arrivals before requeueing the unfinished process.

Example. A's quantum expires at 2 while B waits. C and D arrive at 2, in that input order.

```
At t=2:
Admit C and D      Ready: [B] [C] [D]
Requeue A         Ready: [B] [C] [D] [A]
Dispatch B        CPU: B   Ready: [C] [D] [A]
```

Rule 2. Give the queue head a fresh quantum

When the CPU is free, run the queue head with quantum usage zero. Each execution tick reduces remaining service by one and increases quantum usage by one. New arrivals do not interrupt the turn.

Example. B starts at 0 with service 5 and quantum 2. C arrives at 1.

Time	B's service left	B's quantum used	CPU
0	5	0/2	B
1	4	1/2	B

```
C arrives at 1 -> waits; B runs its second tick.
At 2: B has 3 service ticks left; its turn ends.
```

Rule 3. Complete before considering rotation

If remaining service reaches zero, remove the process, even if its quantum expires at the same boundary. A finished process is never requeued.

Example. A starts at 0 with service 2 and quantum 2. B waits; C arrives at 2.

At $t=2$: A's service = 0 and quantum used = $2/2$

Finish A Ready: [B]

```
Admit C      Ready: [B] [C]
```

```
Dispatch B      CPU: B      Ready: [C]
```

A is finished, so it never rejoins the queue.

Rule 4. Rotate unfinished work when its quantum expires

Append the unfinished process to the queue tail and reset quantum usage. Preserve remaining service. If it is alone, it may immediately start a fresh turn with no idle tick.

Example. A starts at 0 with service 5 and quantum 2.

At $t=2$: A has 3 service ticks left; quantum resets to 0/2.

If B is waiting:

```
Ready before dispatch: [B] [A]  -> B runs next
```

If A is alone:

Turn 1: 0 to 2 Turn 2: 2 to 4 Turn 3: 4 to 5

```
CPU:      A          A          A
                                     -> finished
```

-> finished

Rule 5. Idle only when no process is ready

Wait for the next arrival if the CPU and ready queue are empty. An empty queue does not stop a running process. Stop when all processes finish.

Example. A arrives at 2 with service 1; B arrives at 5 with service 1. Quantum is 2.

Time 0 1 2 3 4 5 6

+-+-+-----+-----+-----+-----+-----+-----+

CPU | - | - | A | - | - | B |

+-+-+-+-----+-----+-----+-----+-----+-----+

Idle from 0 to 2, then again from 3 to 5.

Each process finishes within its first turn.

3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. If an unfinished process has used its quantum, mark its turn as ended.
3. Add all arrivals at the current time.
4. Append the process whose turn ended, with quantum usage reset.
5. If the CPU is free, dispatch the queue head.

A process that has neither finished nor exhausted its quantum continues running.

4. Worked example

Input order is A, B, C. All times are in ticks.

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

Use quantum 2.

```

Time 0   1   2   3   4   5   6   7   8
      +---+---+---+---+---+---+---+---+
CPU   | A | A | B | B | C | A | A | A |
      +---+---+---+---+---+---+---+---+

```

t=2: A's turn ends with 3 service ticks left; B starts.

t=4: B finishes exactly at quantum expiry; C starts.

t=5: C finishes after only 1 tick; A resumes.

t=7: A's turn ends with 1 left; A immediately runs again.

t=8: A finishes; all processes are done.